

Application Threat Model: PASTA Framework Analysis

Project Title: Mobile Sneaker E-Commerce Application Threat Model

Framework: Process of Attack Simulation and Threat Analysis (PASTA)

Target Application: Sneaker Trading & Resale Mobile Application

Executive Summary

This document outlines a threat model completed using the seven-stage **PASTA (Process of Attack Simulation and Threat Analysis)** framework. The analysis evaluates the security posture of a mobile sneaker trading application designed to connect buyers and sellers, process financial transactions, and manage user accounts. By combining architectural reviews, data flow decomposition, and attack tree analysis, key vulnerabilities were identified and mapped to industry-standard security controls

Stages	Sneaker company
I. Define business and security objectives	• Secure E-Commerce & Financial Compliance: Facilitate smooth payment processing across multiple options while adhering strictly to regulatory standards such as PCI-DSS to avoid legal and financial liabilities. • Flexible Account Management & Identity Protection: Allow users to register profiles internally or via external account integrations, ensuring user credential privacy and platform trust. • Platform Availability & Data Privacy: Preserve user privacy across messaging and seller rating features while ensuring continuous availability for searching and inventory interactions.
II. Define the technical scope	• Technologies Evaluated: Application Programming Interface (API), Public Key Infrastructure (PKI - AES/RSA), SHA-256 Hashing, Structured Query Language (SQL). Application Programming Interfaces (APIs) facilitate critical data

	exchange between users, third-party services, and internal systems. Because they expose core functionality to external networks, unmonitored API endpoints create a significant attack surface. Prioritizing API security ensures protection against token abuse, data leakage, and unauthorized systemic access.
III. Decompose application	Sample data flow diagram - Analyzed data movement across core application boundaries, specifically tracing user queries initiated during the Product Search Process through to the backend User Database containing current sneaker inventory. - Identified trust boundaries where raw search inputs from users transition into backend queries, highlighting critical entry points requiring input sanitization and secure token verification.
IV. Threat analysis	Injection Threats (e.g., SQL Injection): External attackers manipulating search inputs or API endpoints to execute unauthorized database commands against the inventory and user databases. Session Hijacking: External threat actors intercepting or exploiting active authentication sessions to unauthorizedly act on behalf of legitimate buyers or sellers.
V. Vulnerability analysis	Lack of Prepared Statements: Dynamic query building in the codebase without parameterization, leaving database handlers open to input manipulation attacks. Broken API Tokens: Weak validation, generation, or transmission practices surrounding API tokens, enabling session hijacking and unauthenticated administrative access.
VI. Attack modeling	Sample attack tree diagram Analyzed high-value targets leading to the compromise of sensitive User Data: Attack Path A (Injection): Lack of prepared statements in database queries —> SQL injection exploit —> Unauthorized access to User Data. Attack Path B (Authentication): Weak login credentials / session management flaws —> Session hijacking —> Impersonation and exposure of User Data.
VII. Risk analysis and	Secure Password Hashing: Use a dedicated

impact	password-hashing algorithm such as Argon2id, with a unique cryptographic salt for each password, rather than using SHA-256 directly. This helps protect stored password credentials against offline brute-force and dictionary attacks. ● Enforced Password Policy & Authentication Security: Enforce strong password complexity rules and Multi-Factor Authentication (MFA) to eliminate weak credentials and protect session tokens. ● Principle of Least Privilege: Enforce strict access control models across APIs and database connections to ensure processes operate strictly with necessary permissions. ● Incident Response Procedures: Establish structured response workflows to quickly detect, contain, and remediate unauthorized access attempts or data exposure event
---------------	--